

Introduction to Cryptography

Lei Xu

lxu12@kent.edu

Computer Science Department

Kent State University

September 26, 2026

Perfect indistinguishability

- Let $\Pi = (\text{Gen}, \text{Enc}, \text{Dec})$ be an encryption scheme with message space $\mathcal{M}$, and let $\mathcal{A}$ be an adversary/algorithm
- Define a randomized experiment $\text{PrivK}_{\mathcal{A}, \Pi}$:
 1. $\mathcal{A}$ outputs $m_0, m_1 \in \mathcal{M}$
 2. $k \leftarrow \text{Gen}, \quad b \leftarrow \{0, 1\}, \quad c \leftarrow \text{Enc}_k(m_b)$
 - c is the *challenge ciphertext*
 3. $b' \leftarrow \mathcal{A}(c)$
- Adversary $\mathcal{A}$ *succeeds* if $b = b'$, and we say the experiment evaluates to 1 in this case

Perfect indistinguishability

- Easy to succeed with probability $\frac{1}{2}$...
- Scheme Π is *perfectly indistinguishable* if for ALL attackers (algorithms) $\mathcal{A}$, it holds that $\Pr[\text{PrivK}_{\mathcal{A},\Pi} = 1] = \frac{1}{2}$

Perfect indistinguishability

- Claim: Π is perfectly indistinguishable $\iff$ Π is perfectly secret
- I.e., perfect indistinguishability is just an alternate definition of perfect secrecy
- We prove that perfect secrecy implies perfect indistinguishability
 - The other direction is a worksheet problem

A useful reformulation

- Perfect secrecy is stated in terms of $\Pr[M = m \mid C = c]$, which depends on the distribution over $\mathcal{M}$
- An equivalent form does not: Π is perfectly secret if and only if for every $m, m' \in \mathcal{M}$ and every $c \in \mathcal{C}$,

$$\Pr[\text{Enc}_K(m) = c] = \Pr[\text{Enc}_K(m') = c]$$

- Probability taken over $k \leftarrow \text{Gen}$ and any coins of Enc
 - I.e., the distribution of the ciphertext does not depend on the message
- One-time pad: $\Pr[\text{Enc}_K(m) = c] = \Pr[K = m \oplus c] = 2^{-n}$, the same for every m

Perfect secrecy $\implies$ perfect indistinguishability

- Let Π be perfectly secret, and let $\mathcal{A}$ be *any* adversary
- In $\text{PrivK}_{\mathcal{A}, \Pi}$, the adversary chooses m_0, m_1 and then sees only $c \leftarrow \text{Enc}_k(m_b)$
- By the reformulation, $\Pr[\text{Enc}_K(m_0) = c] = \Pr[\text{Enc}_K(m_1) = c]$ for every c
 - So c has the *same* distribution whether $b = 0$ or $b = 1$
 - The view of $\mathcal{A}$ carries no information about b
- Hence $b' = \mathcal{A}(c)$ is independent of b , and b is uniform, so $\Pr[b' = b] = \frac{1}{2}$
- This holds for every $\mathcal{A}$, so Π is perfectly indistinguishable

What about the other direction?

- Prove the contrapositive: if Π is not perfectly secret, build an $\mathcal{A}$ that succeeds with probability $> \frac{1}{2}$
- The reformulation gives us m_0, m_1 , and a ciphertext c^* that is more likely under one of them

Computational secrecy?

- Idea: relax perfect indistinguishability
- Two approaches
 - Concrete security
 - Asymptotic security

Computational indistinguishability (concrete)

- (t, ϵ) -indistinguishability:
 - Security may fail with probability $\leq \epsilon$
 - Restrict attention to attackers running in time $\leq t$
 - * Or, t CPU cycles

Computational indistinguishability (concrete version)

- Π is (t, ϵ) -indistinguishable if for all attackers $\mathcal{A}$ running in time at most t , it holds that $\Pr[\text{PrivK}_{\mathcal{A}, \Pi} = 1] \leq \frac{1}{2} + \epsilon$
- Note: $(\infty, 0)$ -indistinguishable = perfect indistinguishability
 - Relax definition by taking $t < \infty$ and $\epsilon > 0$

Concrete security

- Parameters t, ϵ are what we ultimately care about in the real world
- Does not lead to a clean theory...
 - Sensitive to exact computational model
 - Π can be (t, ϵ) -secure for many choices of t, ϵ
- Would like to have schemes where users can adjust the achieved security as desired

Asymptotic security

- Introduce *security parameter* n
 - For now, think of n as the key length
 - Chosen by honest parties when they generate/share key
 - * Allows users to tailor the security level
 - Known by adversary
- Measure running times of all parties, and the success probability of the adversary, as functions of n

Computational indistinguishability (asymptotic)

- Computational indistinguishability:
 - Security may fail with probability *negligible in n*
 - Restrict attention to attackers running in time (at most) *polynomial in n*

Some important definitions

- A function $f : \mathbb{Z}^+ \rightarrow \mathbb{Z}^+$ is *polynomial* if there exists c such that $f(n) < n^c$
- A function $f : \mathbb{Z}^+ \rightarrow [0, 1]$ is *negligible* if for every polynomial p it holds that $f(n) < 1/p(n)$ for large enough n
 - I.e., decays faster than any inverse polynomial
 - Typical example: $f(n) = \text{poly}(n) \cdot 2^{-cn}$

Why these specific choices?

- Somewhat arbitrary
- “Efficient” = “probabilistic polynomial-time (PPT)” borrowed from complexity theory
- Convenient closure properties
 - Poly * poly = poly
 - * A PPT algorithm making calls to PPT subroutines is PPT
 - Poly * negligible = negligible
 - * Poly-many calls to subroutines that fail with negligible probability fail with negligible probability overall

(Re)defining encryption

- A *private-key encryption scheme* is defined by three PPT algorithms (Gen, Enc, Dec):
 - Gen: takes as input 1^n ; outputs k . (Assume $|k| \geq n$.)
 - Enc: takes as input a key k and message $m \in \{0, 1\}^*$; outputs ciphertext c , written $c \leftarrow \text{Enc}_k(m)$
 - Dec: takes key k and ciphertext c as input; outputs a message m or “error”

You may see PPT replaced by “poly-time” or “efficient” in the literature.

Computational indistinguishability (asymptotic version)

- Fix a scheme Π and some adversary $\mathcal{A}$
- Define a randomized exp't $\text{PrivK}_{\mathcal{A},\Pi}(n)$:
 - $\mathcal{A}(1^n)$ outputs $m_0, m_1 \in \{0, 1\}^*$ of equal length
 - $k \leftarrow \text{Gen}(1^n)$, $b \leftarrow \{0, 1\}$, $c \leftarrow \text{Enc}_k(m_b)$
 - $b' \leftarrow \mathcal{A}(c)$
 - Adversary $\mathcal{A}$ *succeeds* if $b = b'$, and we say the experiment evaluates to 1 in this case

Computational indistinguishability (asymptotic version)

- Π is *computationally indistinguishable* if for all PPT attackers $\mathcal{A}$, there is a negligible function ϵ such that $\Pr[\text{PrivK}_{\mathcal{A},\Pi}(n) = 1] \leq \frac{1}{2} + \epsilon(n)$

Example 1

- Consider a scheme where $\text{Gen}(1^n)$ generates a uniform n -bit key, and the best attack is brute-force search of the key space
 - So if $\mathcal{A}$ runs in time $t(n)$, then $\Pr[\text{PrivK}_{\mathcal{A},\Pi}(n) = 1] < \frac{1}{2} + O(t(n)/2^n)$
- Where does that bound come from?
 - Each key trial costs at least a constant, so in time $t(n)$ the attacker tests $O(t(n))$ of the 2^n keys
 - k is uniform, so $\mathcal{A}$ *hits* the real key with probability $q = O(t(n)/2^n)$, and then wins with probability 1
 - If $\mathcal{A}$ misses, the assumption says it learns nothing about b , so it wins with probability $\frac{1}{2}$

$$\Pr[\text{PrivK}_{\mathcal{A},\Pi}(n) = 1] \leq q + (1 - q) \cdot \frac{1}{2} = \frac{1}{2} + \frac{q}{2} = \frac{1}{2} + O(t(n)/2^n)$$

- The advantage over $\frac{1}{2}$ is half the chance the attack works: a failed attack still leaves a coin flip

Example 1 (cont.)

- “The best attack is brute force” is an *assumption* about the scheme
 - Nothing here proves it. For AES we believe it; we cannot prove it
- This scheme is computationally indistinguishable! For any polynomial t , the function $t(n)/2^n$ is negligible
 - Why: fix any polynomial p . Then $t(n) \cdot p(n)$ is again a polynomial, and 2^n outgrows it
 - So $t(n)/2^n < 1/p(n)$ once n is large enough, which is the definition
- Sanity check: the definition accepts a scheme whose only attack is brute force

Example 2

- Consider a scheme and a particular attacker $\mathcal{A}$ that runs for n^3 minutes and breaks the scheme with probability $2^{40} \cdot 2^{-n}$
 - This does not contradict asymptotic security
 - * n^3 is a polynomial, and $2^{40} \cdot 2^{-n}$ is negligible: the constant 2^{40} does not change how fast 2^{-n} decays
 - What about real-world security (against this particular attacker)?
 - * $n = 40$: $\mathcal{A}$ breaks scheme with prob. 1 in 6 weeks
 - * $n = 50$: $\mathcal{A}$ breaks scheme with prob. 1/1000 in 3 months
 - * $n = 500$: $\mathcal{A}$ breaks scheme with prob. 2^{-460} in 200 years
- The definition holds, yet the scheme is broken outright at $n = 40$
 - “Negligible” describes the trend as n grows, not the risk at the n you deploy
 - Choosing n is a concrete-security question (recall slide on (t, ϵ))

Example 3

- What happens when computers get faster?
- E.g., consider a scheme that takes time n^2 to run but time 2^n to break with prob. 1
- What if computers get $4\times$ faster?
 - Honest users double n and can thus maintain the same running time
 - * $(2n)^2 = 4n^2$, and the $4\times$ speedup pays for exactly that
 - Time to break scheme is squared!
 - * Breaking now costs $2^{2n} = (2^n)^2$, and the speedup only divides this by 4
 - * Time required to break the scheme *increases*
- Better hardware helps honest users more than attackers
 - This is what the polynomial vs. exponential gap buys us

Encryption and plaintext length

- In practice, we want encryption schemes that can encrypt arbitrary-length messages
- Encryption does not hide the plaintext length (in general)
 - The definition takes this into account by requiring m_0, m_1 to have the same length
- But beware that leaking plaintext length can often lead to problems in the real world!
 - Obvious examples. . .
 - Database searches
 - Encrypting compressed data

Computational secrecy

- From now on, we will assume the computational setting by default
 - Usually, the *asymptotic* setting

Pseudorandomness

Pseudorandomness

- Important building block for computationally secure encryption
- Important concept in cryptography

What does “random” mean?

- What does “uniform” mean?
- Which of the following is a uniform string?
 - 0101010101010101
 - 0010111011100110
 - 0000000000000000
- If we generate a uniform 16-bit string, each of the above occurs with probability 2^{-16}

What does “uniform” mean?

- “Uniformity” is not a property of a *string*, but a property of a *distribution*
- A distribution on n -bit strings is a function $D : \{0, 1\}^n \rightarrow [0, 1]$ such that $\sum_x D(x) = 1$
 - The *uniform* distribution on n -bit strings, denoted U_n , assigns probability 2^{-n} to every $x \in \{0, 1\}^n$

What does “pseudorandom” mean?

- Informal: cannot be distinguished from uniform (i.e., random)
- Which of the following is pseudorandom?
 - 0101010101010101
 - 0010111011100110
 - 0000000000000000
- Pseudorandomness is a property of a *distribution*, not a *string*

Pseudorandomness (take 1)

- Fix some distribution D on n -bit strings
 - $x \leftarrow D$ means “sample x according to D ”
- Historically, D was considered pseudorandom if it “passed a bunch of statistical tests”
 - $\Pr_{x \leftarrow D}[\text{1st bit of } x \text{ is } 1] \approx \frac{1}{2}$
 - $\Pr_{x \leftarrow D}[\text{parity of } x \text{ is } 1] \approx \frac{1}{2}$
 - $\Pr_{x \leftarrow D}[\text{Test}_i(x) = 1] \approx \Pr_{x \leftarrow U_n}[\text{Test}_i(x) = 1]$ for $i = 1, \dots$

Pseudorandomness (take 2)

- This is not sufficient in an adversarial setting!
 - Who knows what statistical test an attacker will use?
- Cryptographic definition of pseudorandomness:
 - D is pseudorandom if it passes all *efficient* statistical tests

Pseudorandomness (concrete)

- Let D be a distribution on p -bit strings
- D is (t, ϵ) -pseudorandom if for all $\mathcal{A}$ running in time at most t ,

$$\left| \Pr_{x \leftarrow D} [\mathcal{A}(x) = 1] - \Pr_{x \leftarrow U_p} [\mathcal{A}(x) = 1] \right| \leq \epsilon$$

Pseudorandomness (asymptotic)

- Security parameter n , polynomial p
- Let D_n be a distribution over $p(n)$ -bit strings
- Pseudorandomness is a property of a *sequence* of distributions $\{D_n\} = \{D_1, D_2, \dots\}$

Pseudorandomness (asymptotic)

- $\{D_n\}$ is *pseudorandom* if for all probabilistic, polynomial-time distinguishers $\mathcal{A}$, there is a negligible function ϵ such that

$$\left| \Pr_{x \leftarrow D_n} [\mathcal{A}(x) = 1] - \Pr_{x \leftarrow U_{p(n)}} [\mathcal{A}(x) = 1] \right| \leq \epsilon(n)$$

Pseudorandom generators (PRGs)

- A PRG is an efficient, deterministic algorithm that expands a *short, uniform seed* into a *longer, pseudorandom* output
 - Useful whenever you have a “small” number of true random bits, and want lots of “random-looking” bits

PRGs

- Let G be a deterministic, poly-time algorithm that is *expanding*, i.e., $|G(x)| = p(|x|) > |x|$

